

# Funktionsdokumentation — Modellierung

Stand 17.08.2026

## Contents

|                                      |    |
|--------------------------------------|----|
| modelle/config_modelle.py . . . . .  | 2  |
| Modulkopf . . . . .                  | 2  |
| Kommentarbloecke (8) . . . . .       | 2  |
| modelle/m02_menge.py . . . . .       | 4  |
| Modulkopf . . . . .                  | 4  |
| Kommentarbloecke (21) . . . . .      | 5  |
| Funktionen . . . . .                 | 9  |
| modelle/m03_struktur.py . . . . .    | 14 |
| Modulkopf . . . . .                  | 14 |
| Kommentarbloecke (11) . . . . .      | 15 |
| Funktionen . . . . .                 | 16 |
| modelle/m04_shap.py . . . . .        | 20 |
| Modulkopf . . . . .                  | 20 |
| Kommentarbloecke (3) . . . . .       | 21 |
| Funktionen . . . . .                 | 22 |
| modelle/m05_abbildungen.py . . . . . | 26 |
| Modulkopf . . . . .                  | 26 |
| Kommentarbloecke (9) . . . . .       | 28 |
| Funktionen . . . . .                 | 29 |

**modelle/config\_modelle.py**

150 Zeilen · 0 Funktionen

## Modulkopf

Konfiguration der Modellierung. Gegenstueck zu prep/config.py.

DIE TRENNLINIE zwischen beiden Dateien ist nicht "Daten gegen Modelle", sondern:

```
prep/config.py          was in die Parquet-Dateien GESCHRIEBEN wird
modelle/config_modelle.py was nur beim RECHNEN gilt
```

Deshalb stehen Praediktoren, Zielgroessen, Klassen und N\_FOLDS weiterhin in prep/config.py und nicht hier: Sie bestimmen, welche Spalten die Datensaeetze haben und wie die Spalten `fold` und `ist\_holdout` belegt sind. Die Modellskripte LESEN diese Festlegungen, sie treffen sie nicht. Zoege man sie hierher, gaebe es zwei Dateien, die zwingend uebereinstimmen muessen - und genau das ist die Fehlerquelle, die der Aufbau vermeiden soll.

Was hier steht, beruehrt keine einzige Datei auf der Platte.

Bezug: docs/04\_MODELLIERUNG.md

## Kommentarbloecke (8)

### Zeile 21

```
=====
1  HYPERPARAMETER-SUCHE
=====
```

Nur die SUCHRAEUME stehen hier - die Suche selbst laeuft im jeweiligen Modellskript. Ein separates Tuning-Skript wuerde die besten Parameter in eine Datei auslagern, die veralten kann, ohne dass es auffaellt.

Gleiches Budget fuer alle Verfahren -> fairer Vergleich (Bergstra & Bengio 2012 zur Randomized Search, Probst et al. 2019 zu den RF-Raeumen).

WARUM 100 UND NICHT 50 (Decision Log #50, 13.08.2026) - hergeleitet, nicht gewaehlt. Bergstra und Bengio (2012, S. 296) geben die geschlossene Form

$$P = 1 - (1 - v/V)^T$$

fuer die Wahrscheinlichkeit an, mit T Zufallsziehungen mindestens einmal in einen Zielbereich vom relativen Volumen v/V zu treffen. Fuer v/V = 0,05:

```
T = 50  -> 92,3 %
T = 100 -> 99,4 %
```

Der Ausdruck enthaelt die DIMENSION des Suchraums nicht. Genau deshalb bekommen Ridge mit einem und XGBoost mit sieben Hyperparametern dasselbe Budget, ohne dass dem hoeherdimensionalen Verfahren ein Nachteil entsteht - das ist die Antwort auf den naheliegenden Einwand, XGBoost braeuchte mehr.

Die beiden Prozentzahlen sind UNSERE Anwendung ihrer Formel, nicht ihre Aussage. Die verbreitete Angabe "60 Ziehungen" steht nicht in dem Papier (im Volltext geprueft, 13.08.2026); dessen eigene Simulation rechnet mit 1 %.

Der eigentliche Anlass ist #49: Ein weiterer Suchraum verduennt die gute

Region. Wer den Raum oeffnet, muss das Budget mitziehen.

GEMESSEN (tools/suchdiagnose.py, 13.08.2026): Die Verdopplung allein ist bei VIER VON FUENF Verfahren wirkungslos - Random Forest im Mengenstrang gewinnt exakt null, Ridge +0,0065, beide Strukturverfahren +0,0001. Nur XGBoost in der Regression gewinnt spuerbar. Genau so gehoert es in Kapitel 6.

## Zeile 61

```
=====
SUCHRAEUME - vier davon am 13.08.2026 erweitert (Decision Log #49)
=====
```

Erweitert wurde dort, wo der beste gefundene Wert AN der Grenze lag: In sechs von sieben geprueften Parametern lag mindestens ein Fold-Sieger ausserhalb. Die Begrueendung ist eine Aussage ueber die SUCHE, nicht ueber das Ergebnis - das Optimum lag an der Grenze, also war die Grenze falsch gesetzt.

NICHT erweitert wurden max\_features, min\_samples\_leaf, subsample und colsample\_bytree: Ihre Grenzen sind natuerlich (alle Merkmale, eine Beobachtung je Blatt, der ganze Datensatz) - dahinter existiert nichts. Dass der Random Forest dort ans Limit geht, ist ein BEFUND ueber das Verfahren. Ebenfalls nicht erweitert: n\_estimators, nur ein Fold lag nahe der Grenze und es ist der groesste Laufzeittreiber.

## Zeile 77

War  $1e-3$  bis  $1e3$ ; zwei von fuenf Folds lagen ausserhalb (1052 und  $1,1e-05$ ). Ridge ist das einzige Verfahren, das die Baseline gesichert NICHT schlaegt - dort will man die Grenze nicht binden lassen.

## Zeile 84

War [None, 8, 12, 16, 24]. Zwei Aenderungen: nach oben erweitert (Sieger bei 32, 32 und 48), und `None` ans ENDE gestellt. `None` heisst unbegrenzte Tiefe, ist also faktisch der TIEFSTE Wert - an erster Stelle verdrehte es jede Auswertung, die die Listenposition als Tiefe liest (betrifft auch Abbildung A8).

## Zeile 96

War 3 bis 10 - DER wichtigste Fund. Im Strukturstrang waehlten am 07.08. vier von fuenf Folds den Wert 3, also die Untergrenze. Mit geoeffneter Grenze waehlen sie 2, 2 und 1. Das Modell wollte flacher sein, als es durfte. Zusammen mit reg\_lambda ist das die plausibelste Erklaerung fuer R-2: Im Strukturstrang schlagen beide Baumverfahren die Baseline in der Kreuzvalidierung und verlieren auf dem Hold-out, waehrend die Baseline dort BESSER wird - Ueberanpassung, die der Suchraum zum Teil erzwungen hat.

## Zeile 110

Exponent der Tweedie-Varianzfunktion,  $\text{Var} = \mu^p$  (Decision Log #42). 1 waere Poisson, 2 waere Gamma; dazwischen liegt der ueberdisperse Bereich, in dem dieser Datensatz liegt (Dispersionsindex 62,8). Ihn fest auf 1,5 zu setzen waere eine Konvention ohne Grund - also wird er getunt wie jeder andere Hyperparameter.

UNTERGRENZE 1,01 statt 1,1 (#45): Die Baseline ist ein Poisson-GLM, also der Grenzfall  $p = 1$ . Ein Suchraum, der diesen Grenzfall ausschliesst, verbietet XGBoost genau die Loesung, die dem Referenzmodell entspricht - das waere eine Ungleichbehandlung, wie sie #42 und #43 gerade beseitigt haben. `reg:tweedie` verlangt  $1 < p < 2$ , deshalb 1,01 und nicht 1,0. Gilt nur in der REGRESSION; m03 entfernt ihn (Klassifikation).

## Zeile 125

-----  
DIE BASELINES STEHEN HIER NICHT - und zwar aus einem Grund (#45)  
-----

Beide Messlatten sind verallgemeinerte lineare Modelle mit dem fuer die Datenform kanonischen Link, per unpenalisierter Maximum-Likelihood angepasst: Poisson mit Offset fuer die Zaehldaten, multinomiales Logit fuer die nominalen Klassen. Sie haben KEINEN freien Hyperparameter - es gibt nichts zu suchen.

Das ist keine Sparsamkeit gegenueber der Baseline, sondern die Definition von Stufe 2: die einfachste Form, die zur Datenform passt. Ein Strafterm waere eine Erweiterung dieser Form und braechte einen Regler mit sich, den man dann waehlen muesste.

Regel, die daraus folgt und fuer ALLE Modelle gilt: Was einen freien Parameter hat, wird mit demselben Budget getunt. Was keinen hat, wird angepasst. Kein Modell laeuft mit einer unbegruendeten Voreinstellung.

## Zeile 144

=====

2 WIEDERHOLTE SPLITS

=====

Bei 29 Entwicklungsstadtteilen schwankt ein einzelner Fold massiv. Deshalb wird die Fold-Zuteilung mehrfach mit unterschiedlichem Versatz gebildet und ueber alle Laeufer gemittelt (docs/04\_MODELLIERUNG.md, Abschnitt 2).

## modelle/m02\_menge.py

918 Zeilen · 17 Funktionen

## Modulkopf

Verfahrensvergleich fuer die MENGE der Einsatzlast.

Zwei Zielgroessen (`anzahl\_einsaetze`, `einsaetze\_je\_1000\_ew`) x drei Verfahren (Ridge, Random Forest, XGBoost) x 10 Wiederholungen x 5 Folds = 300 Laeufer.

|                                     |                                            |
|-------------------------------------|--------------------------------------------|
| python modelle/m02_menge.py         | Tuning, Bewertung, Aggregation, Vergleich  |
| python modelle/m02_menge.py holdout | zusaeztlich die einmalige Schlussbewertung |

Ausgang: results/regression/menge\_folds.csv · menge\_mittel.csv  
          tuning.csv · vergleich.csv · holdout.csv

Spezifikation: docs/04\_MODELLIERUNG.md. Die dort genannten Fallstricke sind hier im Code markiert - wer eine der vier Stellen aendert, sollte den Abschnitt lesen.

-----

PRUEFAUFTRAEGE nach jedem Lauf

-----

Nachgetragen am 05.08.2026 - dieser Block fehlte, obwohl CLAUDE.md ihn fuer jedes Skript in modelle/ verlangt (docs/07\_BEFUNDE.md, B-9). Abzuarbeiten nach JEDEM Lauf, nicht nur beim ersten.

- 1 Schlaegt jedes Verfahren die Stufe-2-Baseline - je Zielgroesse einzeln? Wenn nein, lautet das Ergebnis "der Mehraufwand lohnt sich hier nicht". Das ist ein Befund, kein Fehler (Gutachten R6).
- 2 Ueberlappen sich die Streuungsbereiche zweier Verfahren? Dann ist "nicht

- unterscheidbar" zu berichten, keine Rangfolge (R-6, R-1).
- 3 Wie oft liefert ein Verfahren NEGATIVE Vorhersagen? Nicht kappen - die Haeufigkeit ist auszuweisen (Spalte n\_negativ). Erwartet: keine, seit Tweedie und Poisson eine Log-Verknuepfung haben (#42, B-15).
  - 4 Passt die Zeilenzahl? 30 in tuning.csv (davon 15 gesucht, 15 zwischen den Zielgroessen geteilt, #43), 300 in menge\_folds.csv.
  - 5 Wurde das Hold-out beruehrt? Ohne Argument darf keine Zeile mit ist\_holdout == 1 gelesen worden sein - main() filtert sie deshalb unwiderruflich heraus, bevor irgendetwas rechnet.
  - 6 Ist std\_wiederholungen deutlich kleiner als std\_folds? Erwartet ja. Waere es null, waeren die Wiederholungen Dubletten (B-3).
  - 7 Steht der Extrapolationsanteil bei rund 34,6 % ueber alle Laeufe? Starke Abweichung heisst, dass die Aufteilung nicht die dokumentierte ist.
  - 8 Laufzeiten: ALLE Verfahren einkernig gemessen, der Parallelisierungs-gewinn getrennt (#39/#40). Kernzahl der Maschine protokollieren.
  - 9 Weicht eine Vorhersage zwischen einkernigem und parallelem Fit ab? Bei XGBoost erwartet (B-24), bei Ridge und RF nicht. Spalte parallel\_abweichung\_max.
  - 10 UEBERANPASSUNG (#51): Wie gross ist `ueberanpassung\_RMSE` je Verfahren? Erwartet: bei Ridge klein, bei den Baumverfahren gross. Der Wert ist ZWISCHEN Konfigurationen zu vergleichen, nicht als Verhaeltnis zwischen Verfahren - Baume interpolieren ihre Trainingsdaten konstruktionsbedingt (Begrueendung an der Fundstelle in `ein\_lauf`).
  - 11 Ist `ueberanpassung\_RMSE` gegenueber der Sicherung vom 07.08. gesunken? Vergleich gegen `archiv/2026-08-14\_budget50/`. NUR FUER `07\_BEFUNDE.md`, nicht fuer die Kapitel: Nach #52 wird ausschliesslich der neue Lauf berichtet, ein Vorher-Nachher-Vergleich ausdruecklich nicht. Faellt die Ueberanpassung NICHT, ist das ein Befund, der die Erklaerung zu R-2 schwaecht - und den man kennen sollte, bevor Kapitel 8 geschrieben wird.

STAND: vollstaendig, 06.08.2026.

## Kommentarbloecke (21)

### Zeile 85

Die Stufe-2-Baseline, gegen die die Primaeraussage laeuft (#34). Der Name muss zu vorpruefung/v1\_baselines.POISSON passen - er wird zum Filtern der Spalte `modell` in baselines\_folds.csv benutzt, ein Tippfehler liefert also stillschweigend eine leere Vergleichsmenge.

### Zeile 91

Der gepaarte Test laeuft auf RMSE. Begrueendung: Bei der Rate ist R2 kein tragfaehiges Mass (docs/03\_STAND.md, Abschnitt 4) - der Mittelwert wird negativ, obwohl die Baseline in jedem Fold besser ist als die Nullmarke. Zwei verschiedene Testmetriken fuer zwei Zielgroessen waeren schwerer zu verteidigen als eine. MAE und R2 wandern als Spalten mit und werden nachrichtlich berichtet.

### Zeile 100

=====

PARALLELISIERUNG - eine Entscheidung mit zwei Gruenden

=====

Die Modelle laufen EINKERNIG, parallelisiert wird nur die Hyperparametersuche.

Grund 1, praktisch: `RandomizedSearchCV(n\_jobs=-1)` um einen Schaetzer mit `n\_jobs=-1` startet Prozesse ueber alle Kerne, von denen jeder seinerseits alle Kerne beansprucht. Die Prozesse blockieren sich gegenseitig; gemessen am 05.08.2026 auf zwei Kernen stand ein Probelauf mit Budget 2 nach 15 Minuten noch in Phase 1 (docs/07\_BEFUNDE.md, B-16).

Grund 2, inhaltlich und wichtiger: Unterfrage 3 fragt nach dem TRAININGS- UND INFERENZAUFWAND. Ridge ist einkernig, weil eine geschlossene Lösung nichts zu parallelisieren hat; RF und XGBoost skalieren ueber Kerne. Misst man sie in unterschiedlichen Betriebsarten, vergleicht man Rechenaufwand und Parallelisierungsgrad in einer Zahl - und die haengt dann an der Kernzahl der Maschine statt am Verfahren.

Deshalb: Der berichtete Aufwand wird EINKERNIG gemessen, fuer alle Verfahren gleich. Der Parallelisierungsgewinn ist eine eigene, ebenfalls interessante Groesse und wird in JEDEM Lauf getrennt miterhoben (siehe `ein\_lauf`).

## Zeile 124

```
=====
EXPOSITION - jedes Verfahren modelliert die RATE (Decision Log #43)
=====

Ein Satz, gueltig fuer alle vier Modelle: geschaetzt wird `einsaetze_je_1000_ew`,
und fuer `anzahl_einsaetze` wird mit der Einwohnerzahl zurueckmultipliziert.
Genau diese Konstruktion verwendet das Poisson-GLM ueber seinen Offset seit
jeher - `vl_baselines.regression()` leitet die Rate ebenfalls aus EINER
Anpassung ab.

WARUM GEAENDERT (06.08.2026, nach dem zweiten Modelllauf):
Der Mechanismustest hat gemessen, was die alte Spezifikation kostete
(docs/07_BEFUNDE.md, B-33): Bei `anzahl_einsaetze` lag Random Forest mit
67,7 gegen 37,4 RMSE hinter der Baseline - modelliert er die Rate und rechnet
zurueck, sind es 36,4. Der gesamte Rueckstand von 24 bis 30 RMSE stammte aus
der Spezifikation, nicht aus dem Verfahren.

Grund fuer die Korrektur, nicht das Ergebnis: Die Forschungsfrage lautet,
welches der drei VERFAHREN die hoechste Prognoseguete erzielt. Verlieren zwei
davon, weil ihnen die Expositionsstruktur vorenthalten wurde, misst der
Vergleich die Modellierungsentscheidung statt der Verfahren. Bei Zaehldaten
mit Expositionsgroesse ist deren explizite Behandlung Standard - XGBoost
bietet dafuer sogar `base_margin`. Sie wegzulassen war ein Fehler derselben
Art wie der quadratische Verlust (#42).

Die Gegenprobe - Baumverfahren OHNE Expositionsbehandlung - ist kein zweiter
Betriebsmodus dieses Skripts, sondern eine eigene Ablation in
`m04_shap.ablation_exposition()`. Hier gibt es keinen Schalter: Der Lauf
hat genau eine Spezifikation.
```

## Zeile 154

```
-----
BAUSTEIN 1 Die Pipeline
-----
```

## Zeile 256

```
-----
BAUSTEIN 2 Das Tuning
-----
```

## Zeile 296

```
-----
BAUSTEIN 3 Ein einzelner Lauf
-----
```

## Zeile 332

```
EXPOSITION (#43): Geschaetzt wird immer die Rate; fuer die absolute Zahl
```

wird mit der Einwohnerzahl zurueckmultipliziert. Dieselbe Konstruktion wie beim Poisson-GLM. Die Zeitmessung bleibt unberuehrt - die Ruecktransformation ist eine Multiplikation und steht ausserhalb.

## Zeile 360

Aendert die Kernzahl das ERGEBNIS? Gemessen statt behauptet - und gemessen statt abgebrochen: Ein Diagnosewert darf einen mehrstuendigen Lauf nicht beenden. Die berichteten Guetemasse stammen ohnehin aus dem einkernigen Fit; der parallele dient allein der Zeitmessung.

BEFUND vom 06.08.2026: Bei XGBoost ist die Abweichung erheblich (gemessen bis 34,7 bei einem Mittelwert von rund 76), bei Ridge und Random Forest null. Ursache ist die parallele Reduktion der Histogramme - eine andere Summierungsreihenfolge kippt knapp benachbarte Split-Kandidaten, und ueber hunderte Baeume schaukelt sich das auf. Das ist eine Aussage ueber die Reproduzierbarkeit von XGBoost und gehoert in Kapitel 6 (docs/07\_BEFUNDE.md, B-24).

## Zeile 374

UEBERANPASSUNGSNACHWEIS, ergaenzt 14.08.2026 (Decision Log #51). Dieselbe Guete auf den TRAININGSstadtteilen. Der Abstand zwischen beiden ist der Standardnachweis fuer Ueberanpassung - ohne ihn bleibt die Diagnose eine Auslegung der Hold-out-Abweichung.

KEIN zweiter Fit: nur eine zusaetzliche Vorhersage auf Daten, die das Modell schon gesehen hat. Sie steht NACH der Zeitmessung, damit Unterfrage 3 unberuehrt bleibt.

Verglichen wird auf der BERICHTETEN Skala: `train[ziel]`, nicht die Groesse, auf der angepasst wurde (das ist immer die Rate, #43).

WIE DIE ZAHL ZU LESEN IST - und wie nicht. Ein Random Forest mit `min\_samples\_leaf = 1` passt die Trainingsdaten KONSTRUKTIONSBEDINGT nahezu perfekt an; jeder Baum interpoliert seine eigene Stichprobe. Ein Trainings-R2 von 0,98 ist dort also erwartbar und nicht schon der Beweis einer krankhaften Ueberanpassung. Der Abstand ist deshalb NICHT als "Verfahren A ueberanpasst 16-mal staerker als B" zu lesen.

Aussagekraeftig ist er in zwei Richtungen:

- zwischen KONFIGURATIONEN desselben Verfahrens (vor und nach der Erweiterung der Suchraeume, #49) - dort ist der Vergleich sauber
- als Groessenordnung gegen die linearen Modelle, die per Konstruktion nicht interpolieren koennen

Der saubere Wert fuer Baeume waere die Out-of-Bag-Schaetzung; sie steht nur beim Random Forest zur Verfuegung und waere gegenueber Ridge und XGBoost asymmetrisch. Bewusst nicht erhoben, hier benannt.

## Zeile 436

-----  
ORCHESTRIERUNG  
-----

## Zeile 466

EXPOSITION (#43): Alle Modelle werden auf der RATE angepasst; fuer `anzahl\_einsaetze` wird die Vorhersage nur zurueckmultipliziert. Es gibt also nur EIN Modell je Verfahren und Fold - und damit auch nur eine Suche. Beide Zielgroessen erhalten denselben Parametersatz, genau wie bei der Baseline, die ebenfalls einmal angepasst wird.

Deshalb laeuft die Suche hier ueber (Verfahren x Fold) = 15 Durchgaenge, und die 30 Zeilen der tuning.csv entstehen erst danach durch Zuordnung zu beiden Zielgroessen. Frueher lief die Schleife ueber die Zielgroessen und die zweite "uebernahm" von der ersten - das Protokoll wies die Suche dann unter `anzahl\_einsaetze` aus, obwohl auf der Rate gesucht wurde (docs/07\_BEFUNDE.md, B-37).

## Zeile 589

Parallelisierungsgewinn: Faktor, um den der Fit ueber alle Kerne schneller ist. Bei Ridge zu erwarten: rund 1 - eine geschlossene Loesung hat nichts zu verteilen. Das ist selbst eine Aussage fuer UF4. Beide Zeiten stammen aus denselben 50 Laeufen.

## Zeile 595

Groesste Abweichung zwischen einkernigem und parallelem Modell. Null heisst threadunabhaengig; alles darueber ist ein Reproduzierbarkeitsbefund und gehoert berichtet (B-24).

## Zeile 602

UEBERANPASSUNG: Trainingsguete und der Abstand zur Testguete. Ein grosser positiver Wert heisst, das Modell erklart die Trainingsstadtteile viel besser als unbekannte - genau das ist Ueberanpassung. Beim Poisson-GLM und bei Ridge ist ein kleiner Abstand zu erwarten, bei den Baumverfahren ein grosser (docs/06\_RISIKEN.md, R-2).

## Zeile 622

-----  
FALLSTRICK 2 Mehrfachvergleiche (R-10)  
-----

## Zeile 704

GEPAART heisst: auf denselben Laeufen. Deshalb wird ueber die Schluessel VERBUNDEN und nicht auf gleiche Reihenfolge vertraut - sonst subtrahiert man stillschweigend verschiedene Testmengen voneinander. Fehlt ein Gegenstueck, bricht der Lauf ab.

## Zeile 739

Holm je Teststufe getrennt, nur innerhalb der sekundaeren Familie. ZWEI FAMILIEN, nicht sieben Tests: Regression und Klassifikation beantworten verschiedene Teilfragen (Entscheidung 05.08.2026, B-6). m03\_struktur.py hat genau einen Test und wird nicht korrigiert.

## Zeile 793

-----  
FALLSTRICK 4 Das Hold-out  
-----

## Zeile 821

DIE BASELINES GEHOEREN DAZU. Ohne sie hat die Schlussbewertung keinen Bezugspunkt: Die Primaeraussage nach #34 lautet "Verfahren gegen Stufe-2-Baseline", und genau die soll der Hold-out pruefen. Ein RMSE von 23,7 ist ohne die Referenz daneben keine Aussage (docs/07\_BEFUNDE.md, B-38). Beide Baselines haben keine Hyperparameter - es gibt nichts zu waehlen und damit auch nichts, was der Hold-out beeinflussen koennte.

## Zeile 874



FALLSTRICK 4, konstruktiv: Ohne das Argument "holdout" wird der Datensatz HIER auf die Entwicklungsstadtteile eingeschränkt. Alles Folgende kann die Hold-out-Zeilen nicht mehr sehen, auch nicht versehentlich.

## Funktionen

**verfahren(name, n\_jobs)**

*Zeile 157 · 65 Zeilen*

Baut die ungetunte Pipeline fuer ein Verfahren.

``n_jobs`` steuert nur die Parallelisierung, nicht das Ergebnis - siehe den Block PARALLELISIERUNG oben. Die Voreinstellung ist EINKERNIG, damit die gemessenen Laufzeiten zwischen den Verfahren vergleichbar bleiben.

FALLSTRICK: Ridge braucht zweierlei, und beides gehoert IN die Pipeline, nicht davor. Der StandardScaler, weil der L2-Strafterm alle Koeffizienten gleich behandelt und Merkmale in verschiedenen Einheiten sonst unterschiedlich hart bestraft wuerden. Und die log-Transformation der ZIELGROESSE ueber TransformedTargetRegressor - der rechnet nach der Vorhersage automatisch mit `expm1` zurueck, sodass die Guetemasse auf der Originalskala entstehen. Wer `log(1+y)` von Hand rechnet, vergisst die Ruecktransformation irgendwann.

Random Forest und XGBoost bekommen keine Zieltransformation: Sie sind gegen Skalen unempfindlich, und eine transformierte Zielgroesse wuerde die Guetemasse zwischen den Verfahren unvergleichbar machen.

VERLUSTFUNKTION (Decision Log #42, 06.08.2026) - eine Korrektur, kein Feintuning. Bis dahin rechneten beide Baumverfahren mit dem QUADRATISCHEN FEHLER auf rohen Zaehldaten, waehrend die Baseline eine Zaehldaten-Likelihood mit log-Verknuepfung benutzte und Ridge auf `log(1+y)` schaeetzte. Zwei Modelle rechneten multiplikativ, zwei additiv - bei Einsatzzahlen von 6 bis 280 und einem Dispersionsindex von 62,8. Der quadratische Fehler gewichtet dort einen Fehler von 20 bei Tenderloin genauso wie bei Seacliff, wo er das Dreifache des Gesamtwerts ausmacht.

Das war eine Ungleichbehandlung in der Spezifikation, nicht ein Ergebnis ueber die Verfahren. Sie ist derselbe Gedanke, mit dem die Negative Binomial als Baseline begruendet wurde: die einfachste Form, die zur DATENFORM passt.

XGBoost ``reg:tweedie``. Die Varianz waechst mit  $\mu$  hoch  $p$ , der Exponent wird getunt (1,1 bis 1,9). Das ist das Gegenstueck zur Negative Binomial, deren Varianz mit  $\mu + \alpha \cdot \mu^2$  waechst; Poisson ( $p = 1$ ) unterstellt Varianz =  $\mu$  und ist bei diesem Dispersionsindex viel zu eng.

RF ``criterion="poisson"``. scikit-learn kennt kein Tweedie fuer Waelder - Poisson ist die naechstgelegene verfuegbare Wahl. Diese Einschraenkung ist selbst ein berichtbarer Befund ueber das Verfahren und gehoert in Kapitel 8, nicht stillschweigend weggelassen.

Ridge unveraendert. ``log(1+y)`` leistet dasselbe bereits.

**suchraum(name)**

*Zeile 224 · 30 Zeilen*

Uebersetzt SUCHRAEUME aus der Config in scipy-Verteilungen.

Die Config haelt die Raeume bewusst als einfache Tupel ("loguniform", a, b), damit sie ohne scipy lesbar bleibt. Hier werden daraus die Objekte, die RandomizedSearchCV erwartet.

Der Praefix haengt am Pipeline-Aufbau: Bei Ridge liegt der Schaetzer zwei Ebenen tief (Pipeline -> TransformedTargetRegressor -> Ridge), bei den Baumverfahren direkt.

**tune(name, train, ziel)**

*Zeile 259 · 35 Zeilen*

Sucht die besten Hyperparameter auf den Trainingsstadtteilen eines Folds.

FALLSTRICK, der die ganze Arbeit entwerfen kann: Der innere CV MUSS nach Stadtteil gruppieren. RandomizedSearchCV nimmt voreingestellt KFold und schneidet zufaellig nach Zeilen - ein Stadtteil hat aber 132 Zeilen, von denen dann etwa 100 im inneren Training und 32 in der inneren Validierung laegen. Da die Strukturmerkmale innerhalb eines Jahres konstant sind, waeren das faktisch dieselben Zeilen: Die Hyperparameter wuerden auf einen geleakten Schaetzwert optimiert, und der Vorteil des aeusseren Stadtteil-Splits waere verspielt. Man sieht es den Zahlen nicht an - sie waeren nur zu gut.

Rueckgabe sind die PARAMETER, nicht das Modell. Wer `best\_estimator\_` weiterverwendet, hat auf dem inneren Trainingsanteil trainiert statt auf allen Trainingsstadtteilen des Folds - und ein Viertel der Daten verschenkt.

ZWEITER FALLSTRICK, seit 06.08.2026 behoben: Der Schaetzer laeuft hier EINKERNIG, parallelisiert wird allein die Suche. Zuvor stand `n\_jobs=-1` an beiden Stellen, und die Prozesse haben sich gegenseitig blockiert (B-16).

**ein\_lauf(name, parameter, train, test, ziel, auch\_parallel)**

*Zeile 299 · 122 Zeilen*

Ein Fit, eine Vorhersage, mit Zeitmessung - eine Zeile fuer die CSV.

FALLSTRICK: Die Zeit wird UM `fit` und `predict` herum gemessen, nicht um die ganze Funktion. Sonst steckt die Metrikberechnung mit in der Zahl, und Unterfrage 3 misst etwas anderes, als sie behauptet.

Gemessen wird EINKERNIG - fuer alle drei Verfahren gleich. Das ist der Aufwand, der in Unterfrage 3 berichtet wird, und er ist zwischen den Verfahren vergleichbar, weil keines einen Parallelisierungsvorteil mitbringt.

Mit `auch\_parallel=True` wird zusaetzlich ein zweiter Fit ueber alle Kerne gemessen. Die Differenz ist der Parallelisierungsgewinn - eine eigene Aussage fuer Unterfrage 4: Ridge hat als geschlossene Loesung nichts zu parallelisieren, die Ensembles skalieren. Im Lauf steht das Argument in JEDEM Aufruf auf True; ein Mass, das nur auf einem Teil der Laeufe beruht, waere eine Ausnahme im Lauf.

Ergaenzt am 05.08.2026 um `n\_negativ` und `y\_hat\_min`: Ridge auf  $\log(1+y)$  kann nach expm1 Werte unter null liefern. Die werden NICHT gekappt - das waere ein Eingriff -, aber ihre Haeufigkeit ist auszuweisen

(docs/04\_MODELLIERUNG.md, Sonderfaelle). Ohne diese zwei Felder muesste man dafuer jedes Modell ein zweites Mal fitten.

### **extrapolationsanteil(train, test)**

#### *Zeile 423 · 11 Zeilen*

Anteil der Testzeilen, die in mindestens einem Merkmal ausserhalb des Trainings-Wertebereichs liegen.

Erklaert spaeter, warum ein Fold aus der Reihe faellt. Erfasst bewusst nur die Spanne je Merkmal, nicht unbekannte KOMBINATIONEN - das echte Extrapolationsproblem ist also eher groesser (docs/06\_RISIKEN.md, R-3).

### **phase\_tuning(panel, selten)**

#### *Zeile 439 · 63 Zeilen*

Je Zielgrosse, Verfahren und Fold einmal `tune()` - 30 Zeilen.

KEINE WIEDERVERWENDUNG. `tuning.csv` ist ein Ergebnis dieses Laufs, kein Eingang. Fruehere Fassungen wurden hier eingelesen, um bei einem Abbruch die teuerste Phase nicht zu verlieren - das hat aber genau die Fehlerklasse erzeugt, die dieses Projekt sonst ueberall vermeidet: Nach einer Aenderung der Spezifikation waeren Parameter aus einer anderen Welt stillschweigend weiterverwendet worden. Ein Lauf, eine Spezifikation, keine Ausnahmen.

Getunt wird ausschliesslich auf Wiederholung 0; die gefundenen Parameter gelten fuer alle zehn Wiederholungen (#34). Das ist eine bewusste Vereinfachung: Die Wiederholungen unterscheiden sich nur in der Fold-Zuteilung und dienen der Streuungsschaetzung, nicht der Modellwahl. Sie ist im Text zu benennen.

Die Parameter landen sowohl als einzelne Spalten (lesbar fuer Kapitel 6.3) als auch als JSON (verlustfrei fuer den Wiedereinlesen-Weg).

ZUR SPALTE `tuning\_sekunden`: Sie steht bei beiden Zielgroessen auf demselben Wert, weil die Suche einmal stattgefunden hat. Eine Summe ueber alle 30 Zeilen zaehlt die Suchzeit deshalb doppelt - die tatsaechliche Dauer von Phase 1 ist die Summe ueber die 15 eindeutigen (Verfahren, Fold)-Paare.

### **\_rein\_python(p)**

#### *Zeile 504 · 17 Zeilen*

NumPy-Skalare in native Typen wandeln, BEVOR sie nach JSON gehen.

Warum das noetig ist: `RandomizedSearchCV.best\_params\_` liefert je nach scipy- und numpy-Fassung `np.int64`/`np.float64` statt `int`/`float`. `np.float64` erbt von `float` und ueberlebt `json.dumps` zufaellig, `np.int64` erbt NICHT von `int`. Mit `default=str` als Notausgang wuerde daraus die Zeichenkette "287", und `set\_params(n\_estimators="287")` bricht ab - mitten im mehrstuendigen Lauf, nach dem Tuning.

Ob es auftritt, haengt an der Paketversion; hier lief es durch, auf einer anderen Kombination nicht zwingend. Deshalb explizit wandeln statt hoffen - und ohne `default=`, damit ein unbekannter Typ laut auffaellt statt still zur Zeichenkette zu werden (docs/07\_BEFUNDE.md, B-23).

## **`_parameter_je_fold(parameter)`**

*Zeile 523 · 5 Zeilen*

```
tuning.csv -> {(zielgroesse, verfahren, fold): dict}.
```

## **`phase_bewertung(panel, parameter, selten)`**

*Zeile 530 · 33 Zeilen*

10 Wiederholungen x 5 Folds x 3 Verfahren x 2 Zielgroessen = 300 Zeilen.

Trainiert wird je Fold auf allen Trainingsstadtteilen - mit den Parametern aus Phase 1, aber einem FRISCHEN Modell. Der ``best_estimator_`` aus dem Tuning waere auf nur drei Vierteln der Trainingsstadtteile gefittet.

## **`aggregiere(folds)`**

*Zeile 570 · 50 Zeilen*

Zweistufig mitteln - erst je Wiederholung, dann darueber.

FALLSTRICK 1 (R-5): Die 50 Fold-Ergebnisse sind NICHT unabhaengig - es sind dieselben 29 Stadtteile in zehn Gruppierungen. Ein Konfidenzintervall aus `std_folds/sqrt(50)` waere deshalb zu eng. Massgeblich ist ``std_wiederholungen``: erst je Wiederholung ueber die 5 Folds mitteln, das ergibt 10 Werte, und deren Standardabweichung wird berichtet.

Beide Spalten wandern mit, damit der Unterschied sichtbar bleibt.

## **`_holm(p)`**

*Zeile 625 · 17 Zeilen*

Holm-Bonferroni: p-Werte aufsteigend, kleinster gegen  $\alpha/m$ , dann  $\alpha/(m-1)$ , bis zur ersten Nichtablehnung.

Zurueckgegeben werden angepasste p-Werte, die direkt gegen  $\alpha$  geprueft werden koennen - das ist dieselbe Entscheidung wie der schrittweise Vergleich, nur bequemer. Uniform staerker als Bonferroni bei gleicher Fehlerkontrolle; es gibt keinen Grund, darauf zu verzichten.

## **`_gepaart(a, b)`**

*Zeile 644 · 22 Zeilen*

Ein gepaarter Wilcoxon plus die Zahlen, die auch ohne p-Wert tragen.

``a`` ist das Verfahren, ``b`` der Gegner. Bei RMSE ist klein besser, die Differenz  $b - a$  ist also der VORTEIL von a.

## **`vergleiche(folds, baselines)`**

*Zeile 668 · 84 Zeilen*

Gepaarter Wilcoxon auf RMSE, zwei Rollen und zwei Teststufen.

ROLLEN

primaer jedes Verfahren gegen die Stufe-2-Baseline ( $3 \times 2 = 6$  Tests).  
KEINE Testfamilie - jede Frage ist nach #34 vorab einzeln formuliert, deshalb keine Korrektur.

sekundaer jedes Verfahrenspaar (3 Paare x 2 Zielgroessen = 6 Tests).  
Eine Familie, darauf Holm-Bonferroni.

TESTSTUFEN (docs/07\_BEFUNDE.md, B-5)

wiederholung n = 10, gemittelt je Wiederholung. DAS IST DER PRIMAERTEST.  
Die 50 Einzellaeufe sind Pseudoreplikation - dieselben 29  
Stadtteile, nur anders gruppiert. Ein Wilcoxon darueber  
liefert zu kleine p-Werte.  
lauf n = 50, alle Einzellaeufe. Ausdruecklich als Sensitivitaet  
gefuehrt, nicht als Ergebnis.

Auch die zehn Wiederholungsmittel sind nicht unabhaengig - es bleiben 29  
Einheiten. Das berichtete Konfidenzintervall ist daher enger als die wahre  
Unsicherheit (Nadeau & Bengio 2003). Deshalb stehen mittlere Differenz,  
Konfidenzintervall und gewonnene Laeufe IMMER daneben, unabhaengig vom p.

### **innere Funktion paar()** *kein Docstring*

#### **leakage\_diagnose(folds, baselines)**

*Zeile 754 · 37 Zeilen*

Beziffert, was das Tuning auf Wiederholung 0 kostet (B-21).

Getunt wird einmal, auf Wiederholung 0. Dort stammen die Parameter aus dem  
Trainingssatz genau dieses Folds - der Vorsprung gegen die Baseline ist  
sauber gemessen. In den Wiederholungen 1 bis 9 werden dieselben Parameter  
auf andere Aufteilungen angewandt; im Mittel waren dort 78 % der  
Teststadtteile in der Menge, auf der die Parameter gesucht wurden.

Waere der Effekt bedeutsam, muesste der Vorsprung in W1-9 SYSTEMATISCH  
groesser ausfallen als in W0. Diese Funktion misst genau das.

Die Diagnose ist bewusst als schwach zu lesen: W0 ist auch eine andere  
Aufteilung als W1-9, der Unterschied ist also konfundiert. Ein deutlicher  
Effekt waere sichtbar, ein kleiner nicht von Fold-Schwankung zu trennen.  
Sie kostet dafuer keine zusaetzliche Rechenzeit - dieselbe Logik, mit der  
R-9 von einem Vorbehalt zu einer Zahl wurde.

#### **hold\_out(panel, parameter, folds, selten)**

*Zeile 796 · 64 Zeilen*

EINMALIG: auf allen 29 Entwicklungsstadtteilen trainieren, auf den 6  
Hold-out-Stadtteilen bewerten.

WELCHE PARAMETER? Das Tuning liefert fuenf Saetze je Zielgroesse und  
Verfahren, einen je Fold. Die Spezifikation sagt "die in der  
Kreuzvalidierung gewaehlten", legt aber nicht fest, welcher davon  
(docs/07\_BEFUNDE.md, B-14). Gewaehlt ist der Satz des Folds mit dem  
niedrigsten RMSE in Wiederholung 0 - deterministisch, nachvollziehbar und  
ausschliesslich aus Entwicklungsdaten. Der gewaehlte Fold steht in der  
Ausgabespalte `fold\_der\_parameter`.

ZU BERICHTEN ist, dass dies EINE Messung an SECHS Einheiten ist - kein  
Mittelwert, keine Streuung. Die Zahl ist deutlich unsicherer als die  
Kreuzvalidierungswerte und darf nicht als deren Bestaetigung gelesen  
werden (R-4).

`main(argv)`

*Zeile 862 · 53 Zeilen*

*kein Docstring*

**modelle/m03\_struktur.py**

*679 Zeilen · 20 Funktionen*

## Modulkopf

Verfahrensvergleich fuer die STRUKTUR der Einsatzlast.

Eine Zielgroesse (`dominante\_einsatzart`, vier Klassen) x zwei Verfahren  
(Random Forest, XGBoost) x 10 Wiederholungen x 5 Folds = 100 Läufe.

|                                                     |                                           |
|-----------------------------------------------------|-------------------------------------------|
| <code>python modelle/m03_struktur.py</code>         | Tuning, Bewertung, Aggregation, Vergleich |
| <code>python modelle/m03_struktur.py holdout</code> | zusätzlich die einmalige Schlussbewertung |

Ausgang: results/klassifikation/struktur\_folds.csv · struktur\_mittel.csv  
          tuning.csv · vergleich.csv · holdout.csv

AUFBAU: Spiegelt m02\_menge.py. Dieselben sieben Funktionen, dieselbe Reihenfolge,  
dieselben Fallstricke. Wer m02 gelesen hat, kennt die Struktur - hier stehen nur  
die Unterschiede.

STAND: vollstaendig, 06.08.2026.

-----  
WAS ANDERS IST ALS IN m02

-----

|             |                                                                                                                                                                                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zielgroesse | `dominante_einsatzart`, vier ungeordnete Klassen                                                                                                                                                                                                    |
| Verfahren   | nur RandomForestClassifier und XGBClassifier - Ridge hat auf<br>einer nominalen Zielgroesse keine Entsprechung (Decision Log #31)                                                                                                                   |
| Guetemasse  | Macro-F1 (Hauptmass) und Macro-AUROC; Accuracy nur nachrichtlich                                                                                                                                                                                    |
| Baseline    | Stufe 2 ist die multinomiale logistische Regression (#33),<br>nicht die Mehrheitsklasse                                                                                                                                                             |
| Scoring     | beim Tuning "f1_macro" statt RMSE                                                                                                                                                                                                                   |
| Holm        | entfaellt. Es gibt genau EINEN sekundaeren Test (RF gegen<br>XGBoost); eine Familie aus einem Test braucht keine Korrektur.<br>Entscheidung vom 05.08.2026: Regression und Klassifikation sind<br>getrennte Testfamilien (docs/07_BEFUNDE.md, B-6). |

-----

DREI FALLSTRICKE, die es in m02 nicht gibt

- 
- 1 KLASSENGEWICHTE statt Resampling. `class\_weight="balanced"` beim Random Forest, `sample\_weight` beim XGBClassifier. KEIN SMOTE, kein Over- oder Undersampling - das waere ein Eingriff in die Datenverteilung und wuerde die Vergleichbarkeit mit den Baselines brechen.
  - 2 LABEL-ENCODER EINMAL GLOBAL fitten, nicht je Fold. XGBClassifier erwartet Integer-Labels 0..3. Wird der Encoder je Fold neu gefittet, verschiebt sich das Mapping in Folds, in denen eine Klasse nicht auftritt - und die Wahrscheinlichkeitsspalten zeigen dann auf die falschen Klassen. Nach der Vorhersage die Spalten auf die Reihenfolge von KLASSEN zurueckbringen.

- 3 MACRO-AUROC KANN UNDEFINIERT SEIN, wenn eine Klasse im Testfold fehlt. Durch die doppelte Stratifizierung (#30) sollte das nicht vorkommen - falls doch, den Wert als FEHLEND fuehren und nicht durch null ersetzen, sonst zieht er den Mittelwert nach unten. `zero\_division=0` bei Macro-F1 muss gesetzt bleiben, sonst bricht der Lauf ab.

-----  
 PRUEFAUFTRAEGE nach jedem Lauf  
 -----

- Schlaegt ueberhaupt ein Verfahren Stufe 2? Wenn nein, ist das ein berichtbares Ergebnis und kein Fehler (docs/06\_RISIKEN.md, R-2).
- Hat jeder Fold Brand-Testfaelle? In Wiederholung 0 erwartet:  $13 \cdot 9 \cdot 6 \cdot 3 \cdot 2$ .
- Liegt Accuracy deutlich ueber Macro-F1? Das ist normal und selbst ein Argument fuer die Metrikwahl - siehe docs/03\_STAND.md.
- Wie viele Läufe haben keine definierte Macro-AUROC? Erwartet: keiner.
- Passt die Zeilenzahl? 10 in tuning.csv, 100 in struktur\_folds.csv.
- Wurde das Hold-out beruehrt? Ohne Argument darf keine Zeile mit `ist_holdout == 1` gelesen worden sein.
- UEBERANPASSUNG (#51): Wie gross ist `ueberanpassung\_macro\_f1`? Dieser Strang ist der, in dem Kreuzvalidierung und Hold-out sich widersprechen (R-2, B-42) - hier entscheidet sich, ob Ueberanpassung die Erklaerung ist.
- Ist der Wert gegenueber `archiv/2026-08-14\_budget50/` gesunken? XGBoost waelte dort vier von fuenf Mal die kleinstmoegliche Baumtiefe. NUR FUER `07\_BEFUNDE.md`: Nach #52 wird kein Vorher-Nachher-Vergleich berichtet. Die Antwort entscheidet aber, wie belastbar die Ueberanpassungserklaerung zu R-2 in Kapitel 8 formuliert werden darf.

## Kommentarbloecke (11)

### Zeile 101

Die vier Klassen in FESTER Reihenfolge - abgeleitet aus ANTEILE, also aus derselben Quelle, aus der die Zielgroesse per argmax entsteht. Diese Liste ist der globale Label-Encoder (Fallstrick 2): Index = Integer-Label.

### Zeile 107

Muss zu vorpruefung/v1\_baselines.LOGREG passen - der Name filtert die Spalte `modell` in baselines\_klasse.csv, ein Tippfehler liefert also stillschweigend eine leere Vergleichsmenge. `hold\_out()` importiert die Konstante direkt.

### Zeile 114

Wie in m02: Modelle einkernig, nur die Suche parallel. Begruendung dort im Block PARALLELISIERUNG (docs/07\_BEFUNDE.md, B-16). Der berichtete Aufwand muss zwischen den Verfahren vergleichbar sein, und der Parallelisierungsgewinn ist eine eigene Groesse.

### Zeile 122

-----  
 BAUSTEIN 1 Die Pipeline  
 -----

### Zeile 205

-----  
 BAUSTEIN 2 Das Tuning  
 -----

### Zeile 240

-----  
BAUSTEIN 3 Ein einzelner Lauf  
-----

## Zeile 277

Anteil der Zeilen, die einkernig und parallel verschieden klassifiziert werden. KEIN Abbruch - gemessen und berichtet: Bei XGBoost ist die Vorhersage threadabhaengig (docs/07\_BEFUNDE.md, B-24). Die berichteten Guetemasse stammen aus dem einkernigen Fit.

## Zeile 283

UEBERANPASSUNGSNACHWEIS, ergaenzte 14.08.2026 - wie in m02, siehe dort. Eine zusätzliche Vorhersage auf den Trainingsstadtteilen, kein zweiter Fit, nach der Zeitmessung. Hier ist der Wert besonders wichtig: Der Strukturstrang ist der, in dem Kreuzvalidierung und Hold-out sich widersprechen (R-2, B-42), und die Baseline auf dem Hold-out BESSER wird, waehrend beide Baumverfahren einbrechen.

## Zeile 370

-----  
ORCHESTRIERUNG  
-----

## Zeile 570

Wie in m02 gehoeren beide Baselines dazu - ohne Bezugspunkt ist ein Macro-F1 von 0,33 keine Aussage (docs/07\_BEFUNDE.md, B-38).

EINE SPEZIFIKATION, ZWEI AUFRUFER (10.08.2026). Bis dahin baute diese Funktion das Logit selbst nach - dieselben vier Argumente, an zwei Orten aufgeschrieben. Aendert jemand eines davon, misst die Kreuzvalidierung still gegen ein anderes Modell als die Schlussbewertung, und keine Pruefung schlaegt an. m02 war immer richtig gebaut und holt `poisson\_glm` aus derselben Datei; hier fehlte genau das.

## Zeile 598

FALLSTRICK 2 auch hier: Die Wahrscheinlichkeitsspalten der logistischen Regression stehen in alphabetischer Reihenfolge ihrer Klassennamen, nicht in der von KLASSEN. Erst umsortieren, dann bewerten - `roc\_auc\_score` verlangt aufsteigend sortierte Labels und liefert sonst gar nichts (B-38).

## Funktionen

`verfahren(name, n_jobs)`

### Zeile 125 · 23 Zeilen

Baut die ungetunte Pipeline. Kein Scaler - beide Verfahren sind Baeume.

`n\_jobs` steuert nur die Parallelisierung, nicht das Ergebnis. Voreinstellung einkernig, damit die Laufzeiten vergleichbar bleiben.

FALLSTRICK 1: Die Klassenverteilung ist stark schief (79 % Fehlalarm). Statt zu resampeln bekommen beide Verfahren GEWICHTE. Beim Random Forest geht das als Hyperparameter (`class\_weight="balanced"`), beim XGBClassifier ueber `sample\_weight` beim Fit - das Verfahren kennt keinen entsprechenden Parameter. Beides bewirkt dasselbe: seltene Klassen zaehlen mehr, ohne dass eine einzige Zeile dupliziert oder geloescht wird.



**suchraum(name)**

*Zeile 150 · 33 Zeilen*

Uebersetzt SUCHRAEUME in scipy-Verteilungen - wie in m02, ohne Praefix.

Beide Verfahren sind hier nackte Schaetzer statt Pipelines, weil keine Skalierung noetig ist. Die Suchraeume sind dieselben wie in der Regression; das ist Absicht: Es wechselt nur die Verlustfunktion, nicht der Ensemble-Mechanismus (docs/04\_MODELLIERUNG.md, Abschnitt 3).

EINE AUSNAHME: `tweedie\_variance\_power` steuert die Verlustfunktion der REGRESSION (Decision Log #42) und ist bei `multi:softprob` bedeutungslos. XGBoost wuerde ihn stillschweigend annehmen und ignorieren - er wuerde dann ein Sechstel des Tuning-Budgets auf eine wirkungslose Dimension verschwenden und in tuning.csv eine Zahl ausweisen, die nichts bedeutet.

**kodierte(y)**

*Zeile 185 · 12 Zeilen*

Klassennamen -> Integer 0..3 nach der GLOBALEN Reihenfolge KLASSEN.

FALLSTRICK 2: Das Mapping haengt bewusst NICHT von den Daten ab, die gerade vorliegen. Ein je Fold gefitteter LabelEncoder wuerde in einem Fold ohne Brand die Zahlen verschieben, und die Wahrscheinlichkeitsspalten zeigten danach auf die falschen Klassen - ohne Fehlermeldung.

**\_gewichte(y\_int)**

*Zeile 199 · 4 Zeilen*

`class\_weight='balanced'` von Hand - fuer XGBoost, das keinen hat.

**tune(name, train)**

*Zeile 208 · 30 Zeilen*

Wie m02.tune, aber mit `f1\_macro` als Scoring.

FALLSTRICK aus m02 gilt unveraendert: Der innere CV MUSS nach Stadtteil gruppieren, sonst stehen dieselben 132 Zeilen eines Stadtteils in innerem Training und innerer Validierung.

Warum f1\_macro und nicht Accuracy: Die Mehrheitsklasse allein erreicht ueber 0,8 Accuracy. Ein darauf optimiertes Tuning wuerde Modelle waehlen, die die drei seltenen Klassen ignorieren - genau das, was die Fragestellung nicht will (docs/03\_STAND.md, Abschnitt 4).

**ein\_lauf(name, parameter, train, test, auch\_parallel)**

*Zeile 243 · 65 Zeilen*

Ein Fit, eine Vorhersage, mit Zeitmessung - eine Zeile fuer die CSV.

Wie in m02 wird die Zeit UM `fit` und `predict` herum gemessen, und zwar EINKERNIG fuer beide Verfahren. Die Wahrscheinlichkeiten fuer die AUROC kommen aus einem zweiten Aufruf, damit `inferenz\_sekunden` die reine Klassenvorhersage misst und zwischen den Verfahren vergleichbar bleibt.

``auch_parallel=True`` misst denselben Fit zusaetzlich ueber alle Kerne.  
Im Lauf steht es in jedem Aufruf auf True - keine Ausnahmen.

### **innere Funktion `fitte()` *kein Docstring***

#### **`extrapolationsanteil(train, test)`**

##### *Zeile 310 · 11 Zeilen*

Anteil der Testzeilen ausserhalb des Trainings-Wertebereichs.

Wortgleich zu `m02_menge`. Bewusst dupliziert statt importiert: Ein gemeinsames Hilfsmodul fuer zwei Aufrufer braechte mehr Indirektion als Ersparnis, und `m03` soll unabhaengig von `m02` lauffaehig bleiben (docs/04\_MODELLIERUNG.md, Abschnitt 4).

#### **`_gepaart(a, b)`**

##### *Zeile 323 · 23 Zeilen*

Ein gepaarter Wilcoxon plus die Zahlen, die auch ohne p-Wert tragen.

Wortgleich zu `m02_menge`; siehe dort. ``a`` ist das Verfahren, ``b`` der Gegner, die Differenz `b - a` ist der Vorteil von `a`. Bei Macro-F1 ist gross besser, die Aufrufstelle dreht die Argumente entsprechend.

#### **`_macro_auroc(y_true, proba, klassen_modell)`**

##### *Zeile 348 · 20 Zeilen*

Macro-AUROC (One-vs-Rest), oder NaN wenn eine Klasse im Test fehlt.

FALLSTRICK 3: NICHT durch 0,5 ersetzen. Ein erfundener Wert saeue wie eine Messung aus und zoege den Mittelwert nach unten. Fehlend heisst fehlend.

``labels=klassen_modell`` bringt die Wahrscheinlichkeitsspalten in die Reihenfolge, die das Modell tatsaechlich benutzt hat - der zweite Teil von Fallstrick 2.

#### **`phase_tuning(panel, selten)`**

##### *Zeile 373 · 21 Zeilen*

Je Verfahren und Fold einmal ``tune()`` auf Wiederholung 0 - 10 Zeilen.

Wie in `m02` wird nichts wiederverwendet: ``tuning.csv`` ist ein Ergebnis dieses Laufs, kein Eingang.

#### **`_rein_python(p)`**

##### *Zeile 396 · 9 Zeilen*

NumPy-Skalare in native Typen wandeln - wortgleich zu `m02_menge`.

``np.int64`` erbt nicht von ``int`` und ueberlebt ``json.dumps`` nicht. Ohne diese Wandlung wuerde aus 287 die Zeichenkette "287", und ``set_params`` braeche nach dem Tuning ab (docs/07\_BEFUNDE.md, B-23).

**\_parameter\_je\_fold(parameter)**

*Zeile 407 · 3 Zeilen*

*kein Docstring*

**phase\_bewertung(panel, parameter, selten)**

*Zeile 412 · 26 Zeilen*

| 10 Wiederholungen x 5 Folds x 2 Verfahren = 100 Zeilen.

**aggregiere(folds)**

*Zeile 445 · 30 Zeilen*

| Zweistufig - wie in m02. Massgeblich ist `std\_wiederholungen` (R-5).

**vergleiche(folds, baselines)**

*Zeile 477 · 50 Zeilen*

| Gepaarter Wilcoxon auf Macro-F1 - zwei primaere Tests, ein sekundaerer.

KEIN HOLM. Die sekundaere Familie besteht aus einem einzigen Test (Random Forest gegen XGBoost); eine Korrektur ueber einen Test ist die Identitaet. Die Spalte `p\_holm` bleibt deshalb leer, `n\_tests\_familie` steht auf 1. Entscheidung vom 05.08.2026: Regression und Klassifikation sind getrennte Testfamilien (docs/07\_BEFUNDE.md, B-6). Das ist in Kapitel 7 zu benennen - dieser Vergleich laeuft ungekorrigiert gegen alpha = 0,05.

Teststufen wie in m02: `wiederholung` (n = 10) ist der Primaertest, `lauf` (n = 50) die ausdruecklich gekennzeichnete Sensitivitaet (B-5).

**innere Funktion paar()** *kein Docstring*

**leakage\_diagnose(folds, baselines)**

*Zeile 529 · 24 Zeilen*

| Beziffert, was das Tuning auf Wiederholung 0 kostet - wie in m02.

Bei Macro-F1 ist GROSS besser, der Vorsprung ist also Verfahren minus Baseline. Ausfuehrliche Begrueendung in `m02\_menge.leakage\_diagnose`.

**hold\_out(panel, parameter, folds)**

*Zeile 555 · 68 Zeilen*

| EINMALIG - wie in m02, mit Macro-F1 statt RMSE als Auswahlkriterium.

Gewaehlt wird der Parametersatz des Folds mit dem HOECHSTEN Macro-F1 in Wiederholung 0. Zu berichten ist, dass dies EINE Messung an SECHS Einheiten ist (R-4).

**main(argv)**

*Zeile 625 · 51 Zeilen*

*kein Docstring*

## modelle/m04\_shap.py

793 Zeilen · 10 Funktionen

### Modulkopf

Interpretation: Welche Merkmale tragen die Vorhersage?

```
python modelle/m04_shap.py
```

Eingang: results/regression/{menge\_folds,vergleich}.csv  
results/klassifikation/{struktur\_folds,vergleich}.csv  
results/\*/tuning.csv · data/processed/{regression,klassifikation}.parquet  
Ausgang: results/shap/beitraege.csv · gruppen.csv · uebersprungen.csv  
faktorgruppen\_menge.csv · vif.csv  
extrapolation\_{merkmale,stadtteile,zusammenhang}.csv  
ablation\_exposition.csv  
ablation\_faktorgruppen.csv · ablation\_faktorgruppen\_mittel.csv

```
python modelle/m04_shap.py --ohne-baeume    Ablation nur fuer GLM und Logit
```

STAND: vollstaendig, 05.08.2026. Faktorgruppen-Ablation ergaenzt 13.08.2026.  
Setzt m02 und m03 voraus.

-----  
ZWEI ANTWORTEN AUF UNTERFRAGE 1 - und warum es beide braucht

-----  
ATTRIBUTION    `gruppen.csv`, `faktorgruppen\_menge.csv`. Welcher Anteil der  
SHAP- bzw. Koeffizientenmasse entfaellt auf eine Faktorgruppe?  
Sagt, wie ein Modell seine Aufmerksamkeit verteilt.

ABLATION        `ablation\_faktorgruppen\_mittel.csv`. Was kostet es, die Gruppe  
wegzulassen? Sagt, was sie WERT ist.

Die zweite Frage ist die haertere: Ein Merkmal kann viel Masse binden und  
trotzdem ersetzbar sein, weil ein anderes dieselbe Information traegt. Erst die  
Ablation trennt das.

-----  
DIE EINE REGEL

-----  
SHAP wird NUR fuer Modelle gerechnet, die ihre Stufe-2-Baseline schlagen. Fuer  
alle anderen erklart man Rauschen - und eine Abbildung, die Beitraege zeigt, wo  
kein Signal ist, ist schlimmer als keine Abbildung. Das Skript prueft das selbst  
und ueberspringt Modelle, die die Latte reissen; die uebersprungenen stehen mit  
Begrueundung in `uebersprungen.csv`, damit die Auswahl nachvollziehbar ist und  
nicht wie Rosinenpicken aussieht.

Massgeblich ist der PRIMAERTEST auf den Wiederholungsmitteln (teststufe  
"wiederholung"): mittlere Differenz zugunsten des Verfahrens UND signifikant.

-----  
WAS GERECHNET WIRD

-----  
TreeExplainer    fuer Random Forest und XGBoost  
Koeffizienten    fuer Ridge - dort braucht es kein SHAP. Der StandardScaler  
steht in der Pipeline, also sind die Koeffizienten bereits  
standardisiert und untereinander vergleichbar.  
Fold             EIN Fold, nicht alle - der mit dem GERINGSTEN  
Extrapolationsanteil in Wiederholung 0. Begrueundung: Dort

liegen die wenigsten Testzeilen ausserhalb des gelernten Wertebereichs, die Beitraege beruhen also am ehesten auf Interpolation. Die Wahl steht in der Ausgabe und ist im Text zu nennen.

#### ----- FALLSTRICK: BLOCKWEISE INTERPRETIEREN -----

Die Strukturmerkmale sind untereinander korreliert. SHAP verteilt den Beitrag dann auf mehrere Merkmale, und einzelne Werte sind nicht sinnvoll deutbar - "median\_haushaltseinkommen traegt 8 %" waere eine Scheinpraezision.

Deshalb zusammenfassen zu den drei Faktorgruppen des Exposés; `log\_bevoelkerung` (Grossenkontrolle) und die Saison werden getrennt ausgewiesen, weil sie in keine der drei Gruppen gehoeren. Das beantwortet Unterfrage 1 direkt: Welche Faktorgruppe traegt wie viel?

#### ----- HIERHER VERSCHOBEN: DER VIF -----

Die Multikollinearitaetspruefung lag frueher in der Eignungspruefung, entschied dort aber nichts - Ridge ist durch den L2-Strafterm robust dagegen, Baumverfahren interessiert sie nicht. Ihre einzige echte Konsequenz betrifft genau diese Interpretation. Deshalb steht sie hier.

Gerechnet auf den EINDEUTIGEN Stadtteil-Merkmalsskombinationen, nicht auf allen Zeilen: Die Strukturmerkmale sind innerhalb eines Jahres konstant, ueber alle Zeilen zaehlte jede Kombination bis zu zweifmal und der VIF waere kuenstlich stabilisiert.

#### ----- PRUEFAUFTRAEGE -----

- Stimmt die Rangfolge der Faktorgruppen zwischen Random Forest und XGBoost ueberein? Wenn nicht, ist das ein Befund fuer Kapitel 8, kein Fehler.
- Passt sie zu den Korrelationen aus der Eignungspruefung? Dort lagen `log_kriminalitaetsindex` und `anteil_risikogewerbe_pct` vorn.
- Wird eine Faktorgruppe als praktisch bedeutungslos ausgewiesen? Das waere eine der wenigen wirklich inhaltlichen Aussagen der Arbeit.
- Liegt der maximale VIF noch bei rund 11,5? Ein deutlich anderer Wert hiesse, dass sich die Merkmalsbasis geaendert hat.
- ABLATION: Reproduziert die Variante `voll` im Mengenstrang exakt die Stufe-2-Baseline aus `results/regression/baselines\_folds.csv`? Wenn nicht, sieht die Ablation andere Merkmale oder andere Folds als v1, und jeder Vergleich darin ist wertlos. Am 13.08.2026 geprueft: Differenz 0,0.
- ABLATION: Welche Gruppen haben ein NEGATIVES Vorzeichen, verbessern die Prognose also durch ihr Weglassen? Das ist ein Befund fuer Kapitel 7 und KEINE Aufforderung, den Merkmalssatz zu kuerzen - er ist durch das Expose und die Fairness-Regel gebunden. Nachtraeglich zu kuerzen waere eine ergebnisgetriebene Spezifikationswahl.

## Kommentarbloecke (3)

### Zeile 320

OHNE Exposition: direkt auf der absoluten Zahl anpassen.  
Der Hauptlauf tut das Gegenteil (Rate schaeetzen, mit der Bevoelkerung zurueckrechnen); die Differenz ist der Effekt.

### Zeile 656

EXPOSITION (#43): Fuer `anzahl\_einsaetze` wurde das bewertete Modell auf der RATE angepasst. Wird hier direkt auf der Anzahl gefittet, erklart SHAP ein anderes Modell als das, dessen Guetemasse berichtet werden - und niemand saehe es den Zahlen an. Die Beitraege beziehen sich also auf das Ratenmodell; das ist im Text zu benennen.

## Zeile 760

--- Ablation der Faktorgruppen (UF1, zweite Antwort) ---  
Attribution sagt, wie ein Modell seine Aufmerksamkeit verteilt.  
Diese Ablation sagt, was die Gruppe wert ist. Siehe Docstring dort.

## Funktionen

### `schlagen_die_latte(vergleich)`

#### Zeile 142 · 21 Zeilen

Welche (Zielgroesse, Verfahren) schlagen ihre Stufe-2-Baseline?

Grundlage ist der Primaertest auf den Wiederholungsmitteln. Verlangt werden BEIDE Bedingungen: die mittlere Differenz muss zugunsten des Verfahrens ausfallen UND der Test muss signifikant sein. Ein positiver Mittelwert allein waere zu wenig - genau davor warnt R-6.

### `ruhigster_fold(folds)`

#### Zeile 165 · 5 Zeilen

Der Fold mit dem geringsten Extrapolationsanteil in Wiederholung 0.

### `_beitraege(modell, X, name)`

#### Zeile 172 · 36 Zeilen

Mittlerer absoluter Beitrag je Merkmal - SHAP oder Koeffizient.

Bei Ridge stehen standardisierte Koeffizienten; sie sind der direkte Gegenwert zu SHAP-Beitraegen und brauchen keinen Explainer. Bei den Baumverfahren rechnet der TreeExplainer exakt statt zu approximieren.

Mehrklassige Ausgaben werden ueber die Klassen gemittelt - die Frage lautet "welche Faktorgruppe traegt", nicht "fuer welche Klasse".

WARUM XGBOOST EINEN EIGENEN WEG GEHT: `shap.TreeExplainer` kann den mehrklassigen `base\_score` von XGBoost 3.x nicht lesen und bricht mit `could not convert string to float` ab (geprueft mit shap 0.52.0 und xgboost 3.2.0, docs/07\_BEFUNDE.md, B-17). XGBoost bringt TreeSHAP aber selbst mit - `pred\_contribs=True` liefert exakt dieselben Werte, gerechnet vom selben Algorithmus. Kein Naeherungsverfahren, nur ein anderer Aufrufweg.

### `extrapolation_aufschluesseln(panel, selten, folds)`

#### Zeile 210 · 57 Zeilen

Woher kommen die 33,7 % Extrapolation - und was folgt daraus?

WARUM DAS HIER STEHT: `03\_STAND.md` behauptet, die Spanne des Extrapolationsanteils von 3,6 % bis 57,4 % erklare „einen erheblichen Teil der Fold-Streuung“. Das war eine Plausibilitaetsaussage ohne Messung. Diese

Funktion macht eine Zahl daraus. Sie erklart damit den zentralen Befund des Mengenstrangs (R-3, `07\_BEFUNDE.md` B-26) und gehoert deshalb zur Interpretation, nicht zum Verfahrensvergleich.

ABGRENZUNG ZU #34 - wichtig, das ist keine Haarspalterei:

Verboten ist, die TESTMENGE nach Extrapolationsgrad aufzuteilen und dort nach Verfahrensunterschieden zu suchen; das waere ein nachtraeglicher Zuschnitt der Auswertung. Hier wird nichts aufgeteilt und nichts neu verglichen. Die Einheit ist der FOLD, und die Frage lautet, warum Folds unterschiedlich schwer sind. Die Primaeraussage bleibt unberuehrt.

Drei Auswertungen:

- 1 je Merkmal wie oft liegt es allein ausserhalb des Trainingsbereichs
- 2 je Stadtteil wie stark bricht er aus, wenn er im Test steht
- 3 je Verfahren Zusammenhang zwischen Extrapolationsanteil eines Laufs und dem dort gemessenen Fehler (Spearman, ueber alle 50 Laeufe)

**ablation\_exposition(panel, selten, parameter)**

*Zeile 269 · 66 Zeilen*

ABLATION: Was leistet die Expositionsbehandlung?

Der Hauptlauf modelliert die Rate und multipliziert mit der Einwohnerzahl zurueck (#43) - fuer alle vier Modelle gleich. Diese Ablation entfernt genau diesen einen Baustein bei den Baumverfahren und laesst sie direkt auf `anzahl\_einsaetze` anpassen. Alles andere bleibt identisch: dieselben Folds, dieselben Merkmale, dieselben Hyperparameter.

Es wird also EIN Bestandteil der Spezifikation isoliert. Das ist der Zweck einer Ablation und der Grund, warum die Hyperparameter bewusst NICHT neu gesucht werden - sonst aenderte man zwei Dinge gleichzeitig.

WAS SIE BEANTWORTET. Unterfrage 4 fragt nach Implikationen fuer die Modellauswahl. Die Ablation zeigt, ob die Wahl des Verfahrens oder die Spezifikation den groesseren Hebel hat - und liefert damit eine uebertragbare Aussage statt eines knappen Rankings.

Frueher gemessen (`07\_BEFUNDE.md`, B-33): Ohne Expositionsbehandlung lagen Random Forest bei 67,7 und XGBoost bei 61,7 RMSE, mit ihr bei 36,4 und 35,7. Der Unterschied zwischen den Spezifikationen eines Verfahrens ist damit ein Vielfaches des Unterschieds zwischen den Verfahren.

DIE FRAGE. Bei `anzahl\_einsaetze` liegen die Baumverfahren rund 20 RMSE hinter Ridge, bei `einsaetze\_je\_1000\_ew` leicht davor. Der einzige Unterschied zwischen beiden Zielgroessen ist die Einwohnerzahl. Die Vermutung lautet: Baeume koennen „Einsaetze = Bevoelkerung x Risiko“ nicht nachbauen, weil sie je Blatt einen festen Wert ausgeben und Extremwerte zur Blattmitte ziehen – und weil RMSE auf der Originalskala von den grossen Stadtteilen dominiert wird (Tenderloin 280, Seacliff 6,4).

Spiegelbild zu R-9: Dort wurde der Offset der Baseline WEGGENOMMEN, Ergebnis null. Hier fehlt er den Baeumen. Kein Widerspruch – fuer ein Modell mit Log-Verknuepfung und freiem Koeffizienten auf `log\_bevoelkerung` ist der Offset redundant, fuer einen Baum ohne beides nicht.

**ablation\_faktorgruppen(reg, kl, selten, tuning\_kl, mit\_baeumen)**

*Zeile 337 · 117 Zeilen*

UNTERFRAGE 1, zweite Antwort: Was ist eine Faktorgruppe WERT?

WARUM ES DIESE FUNKTION BRAUCHT. `beitraege.csv` und `gruppen.csv` beantworten Uf1 ueber ATTRIBUTION - welcher Anteil der Koeffizienten- bzw. SHAP-Masse auf eine Gruppe entfaellt. Das sagt, wie ein Modell seine Aufmerksamkeit verteilt. Es sagt NICHT, was die Gruppe wert ist: Ein Merkmal kann viel Masse binden und trotzdem ersetzbar sein, weil ein anderes dieselbe Information traegt.

Die Ablation misst das Fehlende direkt. Jede Gruppe wird einmal weggelassen, alles andere bleibt gleich - dieselben Folds, dieselben Zeilen, dieselbe Spezifikation. Die Verschlechterung ist der Beitrag.

Dasselbe Muster wie `ablation\_exposition()`, nur auf die Merkmalsgruppen statt auf die Expositionsbehandlung angewandt.

WELCHES MODELL JE STRANG - nach derselben Regel wie der Rest von m04: abladiert wird das Modell, dessen Beitraege berichtet werden.

|          |                                                                                                                                                                                                                                 |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Menge    | das Poisson-GLM. Kein Vergleichsverfahren schlaegt es (B-26), es IST das beste Modell des Strangs. Und es hat keinen Hyperparameter - die Ablation ist dadurch sauber: Was sich aendert, ist ausschliesslich die Merkmalsmenge. |
| Struktur | Random Forest und XGBoost. Beide schlagen die Stufe-2-Baseline in der Kreuzvalidierung (B-29), fuer beide werden SHAP-Beitraege berichtet. Das Logit laeuft zum Vergleich mit.                                                  |

EINE EINSCHRAENKUNG, die zu berichten ist: Bei den Baumverfahren stammen die Hyperparameter aus dem VOLLEN Merkmalssatz und werden nicht neu gesucht - genau wie in `ablation\_exposition()`, damit sich nur EIN Ding aendert. Die gemessene Verschlechterung enthaelt dadurch einen Anteil, der auf eine nicht mehr passende Einstellung entfaellt und nicht auf die fehlende Information. Beim Poisson-GLM besteht dieses Problem nicht.

KEIN SIGNIFIKANZTEST. Die Testfamilien sind mit #38 festgelegt - zwei, eine je Strang. Weitere Tests hier wuerden die Korrekturstruktur beruehren und muessten in Holm eingehen. Die Ablation ist deskriptiv gemeint: berichtet werden Mittelwert, Streuung ueber die zehn Wiederholungsmittel (R-5) und die Zahl der Wiederholungen, in denen die Gruppe fehlte.

Der Offset des Poisson-GLM bleibt in JEDER Variante bestehen, auch wenn die Groessenkontrolle weggelassen wird: `log(Bevoelkerung)` geht als Offset ein, nicht als Merkmalsspalte. Weggelassen wird nur der Praediktor `log\_bevoelkerung`.

## \_ablation\_auswerten(roh)

*Zeile 456 · 35 Zeilen*

Verschlechterung je Gruppe gegenueber dem vollen Merkmalssatz.

Gepaart je Lauf: Die Variante und der volle Satz laufen auf demselben Fold derselben Wiederholung. Gemittelt wird zweistufig - erst je Wiederholung ueber die Folds, dann darueber (R-5), weil die 50 Laeufer nicht unabhaengig sind.

Das VORZEICHEN ist so gedreht, dass ein positiver Wert immer "Verschlechterung durch Weglassen" heisst - bei RMSE ist klein besser, bei Macro-F1 gross. Ohne diese Drehung liest man eine der beiden Tabellen



genau falsch herum.

### **faktorgruppen\_baseline(panel, selten, fold)**

#### *Zeile 493 · 50 Zeilen*

Beitrag der drei Faktorgruppen im MENGENSTRANG - aus der Baseline.

WARUM AUS DER BASELINE. Unterfrage 1 fragt nach dem Erklärungsbeitrag der drei Faktorgruppen. Fuer die Struktur liefert ihn SHAP. Fuer die Menge nicht: `m04` ueberspringt dort alle Modelle, weil keines seine Baseline schlaegt – und Beitraege eines unterlegenen Modells auszuweisen hiesse, Rauschen zu erklaren.

Die Loesung liegt im Ergebnis selbst: Das **\*\*beste Modell des Mengenstrangs** ist das Poisson-GLM\*\*. Seine Koeffizienten beantworten UF1 direkt und ehrlich. Dass die Antwort aus der Baseline statt aus einem Vergleichsverfahren kommt, ist kein Notbehelf, sondern die Konsequenz des Befunds.

VERGLEICHBAR GEMACHT ueber standardisierte Beitraege  $|\text{Koeffizient}| \times \text{Standardabweichung des Merkmals}$ . Ohne diesen Schritt haengt die Groesse eines Koeffizienten an der Einheit des Merkmals – Einkommen in Dollar bekaeme automatisch einen winzigen Koeffizienten.

Gerechnet auf demselben Fold wie die SHAP-Werte, damit beide Straenge dieselbe Datengrundlage haben.

### **\_vif(panel)**

#### *Zeile 545 · 38 Zeilen*

VIF auf zwei Bezugsmengen - und der Grund, warum es zwei sein muessen.

Die Absicht der Spezifikation war, jede Merkmalskombination nur EINMAL zu zaehlen: Die Strukturmerkmale sind innerhalb eines Jahres konstant, ueber alle Zeilen zaehlte jede Kombination bis zu zweelfmal, und der VIF waere kuenstlich stabilisiert.

Ein `drop\_duplicates()` auf allen Praediktoren leistet das aber NICHT: Seit Decision Log #17 ist `log\_kriminalitaetsindex` ein MONATLICH rollierender Index. Damit ist fast jede Zeile eindeutig - gemessen 3.757 von 3.828 - und die Entdopplung laeuft ins Leere (docs/07\_BEFUNDE.md, B-18).

Deshalb zwei ausgewiesene Bezugsmengen:

|                |                                                                                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| stadtteil_jahr | eine Zeile je Stadtteil und Jahr. Das ist die Ebene, auf der die ACS- und Land-Use-Merkmale tatsaechlich variieren, und die Zahl, die in den Text gehoert. |
| alle_zeilen    | zum Vergleich, damit der Unterschied sichtbar ist.                                                                                                         |

### **main()**

#### *Zeile 585 · 205 Zeilen*

Sechs Auswertungen zu Unterfrage 1, in dieser Reihenfolge.

1 **ATTRIBUTION**. Fuer jede Kombination aus Zielgroesse und Verfahren, die `schlagen\_die\_latte()` durchlaesst, werden auf dem ruhigsten Fold SHAP-Beitraege berechnet und zu Faktorgruppen verdichtet

- > beitraege.csv, gruppen.csv, uebersprungen.csv.
- Schlaegt kein Modell seine Baseline, bleibt der Block leer - das ist ein Ergebnis, kein Fehler (R-2). Genau das ist im Mengenstrang der Fall.
- 2 EXTRAPOLATION aufgeschlüsselt nach Merkmal und Stadtteil, plus der Zusammenhang zum RMSE -> extrapolation\_\*.csv.
- 3 ABLATION DER EXPOSITION: dieselben Baumverfahren ohne die Ruecktransformation ueber die Einwohnerzahl -> ablation\_exposition.csv.
- 4 FAKTORGRUPPEN DES MENGENSTRANGS aus der Baseline, weil dort Schritt 1 leer bleibt -> faktorgruppen\_menge.csv.
- 5 ABLATION DER FAKTORGRUPPEN: was kostet das Weglassen einer Gruppe -> ablation\_faktorgruppen.csv, ablation\_faktorgruppen\_mittel.csv.
- 6 VIF als Kollinearitaetsmass -> vif.csv.

Das Hold-out wird vor Schritt 1 herausgefiltert und nie wieder angefasst; dieses Skript kennt kein "holdout"-Argument. Die Schritte 3 bis 5 sind Zusatzbelege und beruehren den Verfahrensvergleich nicht - sie erklæaren ihn nur. Mit `--ohne-baeume` laeuft Schritt 5 nur auf den GLM-Baselines, was die Laufzeit von rund zehn auf unter eine Minute drueckt.

## modelle/m05\_abbildungen.py

1129 Zeilen · 21 Funktionen

## Modulkopf

Alle Abbildungen fuer Kapitel 7 - aus den CSV-Dateien, nicht von Hand.

```
python modelle/m05_abbildungen.py
```

Eingang: results/regression/\*.csv · results/klassifikation/\*.csv  
 results/spezifikation/\*.csv  
 results/shap/{ablation\_exposition,gruppen,faktorgruppen\_menge,  
 extrapolation\_zusammenhang}.csv

Ausgang: results/abbildungen/\*.pdf

STAND: neu gefasst 07.08.2026, um A6-A9 erweitert 10.08.2026. Setzt m02, m03, m04, v1 und v3 voraus.

Dieses Skript RECHNET NICHTS. Es liest nur. Dadurch laesst sich eine Darstellung aendern, ohne die Modelle neu zu rechnen, und nach einem neuen Lauf ist ein Befehl genug.

-----  
 WARUM DER SATZ AM 07.08.2026 NEU GESCHNITTEN WURDE  
 -----

Der erste Satz bestand aus Boxplots der Rohwerte je Verfahren und einem Balkendiagramm gegen die Baseline. Beides zeigte den Vergleich nicht, und zwar aus einem messbaren Grund.

Die 50 Laeufe unterscheiden sich darin, WELCHE Stadtteile im Testfold liegen. Bayview hat ein Vielfaches der Einsaetze von Seacliff, also schwankt der RMSE zwischen 13 und 76 - unabhæangig vom Verfahren. Die Streuung der Rohwerte betraegt 12,4 bis 15,5 RMSE, der Verfahrensunterschied rund 2. Ein Boxplot der Rohwerte zeigt daher fast ausschliesslich Fold-Streuung.

Jedes Verfahren sieht aber DIESELBEN Folds. Bildet man die Differenz je Lauf, kuerzt sich die Fold-Streuung heraus: Die Streuung der gepaarten Differenz ueber die 10 Wiederholungsmittel betraegt 2,4 bis 4,3. Gepaarte Daten ungepaart darzustellen verschenkt genau die Information, fuer die das Design gebaut wurde

- und es ist dieselbe Paarung, auf der der Wilcoxon-Test beruht (#34).

Das alte A2 hatte zusätzlich einen einfachen Darstellungsfehler: Balken ab null, während sich alles zwischen 33,98 und 36,51 abspielt. Die Unterschiede lagen in den obersten sechs Prozent der Bildhöhe.

---

#### ZEHN ABBILDUNGEN

---

A1 bis A5 tragen den Verfahrensvergleich, A6 bis A9 die Interpretation, A10 die Annahmenprüfung. Alle zehn lesen ausschliesslich CSV - die Regel "dieses Skript rechnet nichts" gilt unverändert auch fuer die fuenf neuen.

|                         |                                                                                                                                                                                                                |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| a1_gegen_baseline.pdf   | Gepaarte Differenz zur Stufe-2-Baseline, ein Punkt je Wiederholung, beide Straenge nebeneinander. Das ist die Primaeraussage nach Decision Log #34.                                                            |
| a2_foldstruktur.pdf     | Die Rohwerte je Fold, Verfahren als Linien. Zeigt, dass die Streuung aus dem Fold stammt und nicht aus dem Verfahren - die Begrueundung fuer A1.                                                               |
| a3_spezifikation.pdf    | Was bewegt mehr: die Wahl des Verfahrens oder die Wahl der Spezifikation? Antwort auf Unterfrage 4, Grundlage von B-41.                                                                                        |
| a4_laufzeit_guete.pdf   | Trainingszeit gegen Prognoseguete, ein Punkt je Verfahren. Unterfrage 3.                                                                                                                                       |
| a5_holdout.pdf          | Die einmalige Auswertung auf den sechs zurueckgehaltenen Stadtteilen, beide Straenge.                                                                                                                          |
| a6_faktorgruppen.pdf    | Welche der drei Faktorgruppen des Exposés traegt wie viel? Das ist UNTERFRAGE 1 - die einzige der vier, zu der es bisher keine Abbildung gab, sondern nur Konsolenausgabe von m04.                             |
| a7_extrapolation.pdf    | Extrapolationsanteil eines Laufs gegen den dort gemessenen Fehler, 50 Punkte je Verfahren, mit Spearman-rho. Macht R-3 sichtbar und liefert die Begrueundung, die A2 nur behauptet.                            |
| a8_hyperparameter.pdf   | Wie stabil ist die Modellwahl? Die fuenf Fold-Parametersaetze je Verfahren, jeder auf seine Lage im eigenen Suchraum normiert. Grundlage fuer Kapitel 8.                                                       |
| a9_parallelisierung.pdf | Parallelisierungsgewinn je Verfahren. Zweite Haelfte von Unterfrage 3, die A4 nicht zeigt: A4 traegt die EINKERNIGE Zeit auf, hier steht, was Kerne bringen - und wo sie nichts bringen.                       |
| a10_qq_residuen.pdf     | QQ-Diagramm der Residuen der linearen Spezifikation. Auflage vom 10.08.2026, gehoert zu Abschnitt 6 der Eignungsprüfung. Zeigt, WO die Verteilung von der Normalverteilung abweicht - der Test sagt nur, DASS. |

---

#### ANFORDERUNGEN AN DIE DARSTELLUNG

---

Sie landen im gedruckten Dokument, und Gestaltung war im Gutachten ein eigenes Bewertungskriterium.

|            |                                                                                                                                                                            |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Format     | PDF, nicht PNG. Rasterbilder werden im Druck unscharf.                                                                                                                     |
| Groesse    | In der ENDGROESSE erzeugen, nicht gross erzeugen und in LaTeX schrumpfen - sonst steht dort 5-pt-Schrift. Mindestens 9 pt.                                                 |
| Titel      | KEINE Titel in der Abbildung. Die Bildunterschrift in LaTeX ist der Titel; beides doppelt sich sonst.                                                                      |
| Graustufen | Verfahren zusätzlich ueber Schraffur und Marker unterscheiden, nicht allein ueber Farbe.                                                                                   |
| Achsen     | Beschriftung mit Einheit, deutsches Dezimalkomma.                                                                                                                          |
| Nulllinie  | Wo eine Differenz oder ein R <sup>2</sup> dargestellt wird, ist sie einzuzeichnen - das Vorzeichen ist die Aussage.                                                        |
| Richtung   | Bei jeder Differenzachse muss dastehen, welche Seite besser ist. Bei RMSE ist das links, bei Macro-F1 rechts - wer das verwechselt, liest das Ergebnis genau falsch herum. |
| Streuung   | IMMER benennen, worueber sie gebildet ist: ueber die 10 Wiederholungsmittel, nicht ueber die 50 Einzelläufe (R-5).                                                         |

#### ----- PRUEFAUFTRAEGE

- Sind alle fuenf PDF entstanden und in LaTeX einbindbar?
- Schneidet in A1 die Nulllinie eine der Boxen? Dann darf im Text kein Unterschied zur Baseline behauptet werden, den der Test nicht deckt (R-6).
- Traegt jede Differenzachse die Richtungsangabe, und zeigt sie bei Macro-F1 in die andere Richtung als bei RMSE?
- Stimmt der Referenzwert in A3 mit `linear` aus v3 und mit der Stufe-2-Baseline aus v1 ueberein? Alle drei muessen dieselbe Zahl sein.
- In Graustufen ausdrucken: sind die Verfahren noch unterscheidbar?
- A6: Summiert sich jeder Balken auf 100 %? Steht in der Fusszeile, dass der Mengenbalken KOEFFIZIENTEN und die Strukturbalken SHAP-Werte zeigen? Die beiden Groessen sind nicht dasselbe und duerfen nicht als eine gelesen werden.
- A7: Liegen die drei Verfahren bei gleichem x uebereinander? Muessen sie - der Extrapolationsanteil ist eine Eigenschaft des Folds, nicht des Verfahrens. Andernfalls stimmt die Fold-Zuordnung nicht.
- A8: Klebt ein Parameter am Rand seines Suchraums (Lage nahe 0 oder 1)? Dann war der Raum zu eng gewaehlt, und das gehoert in die Limitationen.
- A9: Steht die Linie bei 1,0 und ist beschriftet? Werte UNTER 1 heissen "parallel langsamer" - ohne die Linie liest man sie als Gewinn.

## Kommentarbloecke (9)

### Zeile 175

-----  
Nur fuer A6 bis A9  
-----

Reihenfolge der Faktorgruppen: die drei des Exposés zuerst, danach die zwei getrennt gefuehrten Groessen. Identisch zu GRUPPEN in m04\_shap.py - stuenden sie in anderer Reihenfolge, waeren Abbildung und Tabelle nicht vergleichbar.

### Zeile 184

Helle Fuellung plus eigene Schraffur je Gruppe. In gestapelten Balken traegt die Schraffur die Unterscheidung, nicht der Grauwert - fuenf Grautoene sind im Druck nicht mehr sicher auseinanderzuhalten.

### Zeile 539

Die Stufe-2-Baseline als Linie. Ohne sie ist der sehr enge Wertebereich nicht einzuordnen - 0,9 RMSE Unterschied saehen aus wie ein Abgrund, obwohl alle drei Verfahren dicht an der Latte liegen.

## Zeile 610

Die Baselines heissen in der CSV so, wie sie in Kapitel 5 heissen -  
ausgeschrieben. Als Achsenbeschriftung sind sie zu lang und  
ueberdecken die Nachbarfelder; hier stehen die Kurzformen.

## Zeile 708

Zahl nur, wo das Segment sie traegt. Ein Label, das in den  
Nachbarn ragt, ist schlimmer als kein Label. Der weisse Kasten  
dahinter ist noetig, weil die Schraffur sonst durch die Ziffern  
laeuft - im Druck sind sie dann nicht mehr sicher lesbar.

## Zeile 851

In der Regression steht jeder Suchlauf ZWEIMAL in der Datei, einmal  
je Zielgroesse - gesucht wurde aber nur einmal, auf der Rate (#43).  
Ohne diese Entdopplung stuenden hier zehn statt fuenf Punkte je  
Parameter, und die Streuung saehe nur halb so gross aus.

## Zeile 890

Menge vor Struktur, innerhalb dessen die Verfahrensreihenfolge der Arbeit  
(Ridge, Random Forest, XGBoost) - nicht die alphabetische, in der Ridge  
zwischen den Ensembles stuende.

## Zeile 898

Y-Positionen mit LUECKE zwischen den Straengen. Ohne die Luecke bliebe  
kein Platz fuer die Blockueberschriften, und ohne die Ueberschriften ist  
nicht ablesbar, welcher Block welcher ist - die Verfahrensnamen  
wiederholen sich in beiden.

## Zeile 978

Im Mengenstrang stammen beide Zielgroessen aus DERSELBEN Anpassung  
(#43) - fuer `anzahl\_einsaetze` wird die Ratenvorhersage nur  
zurueckmultipliziert. Die Zeiten sind deshalb bis auf Messrauschen  
identisch; zwei Balken taeuschten zwei Messungen vor.

# Funktionen

## `_sekunden(wert)`

### *Zeile 216 · 8 Zeilen*

Sekunden lesbar beschriften.

Zwei Nachkommastellen reichen fuer die Ensembles (5,83 s), nicht fuer Ridge  
(0,011 s) - dort stuende sonst zweimal "0,01 s" und die Abbildung  
behauptete, der parallele Fit sei gleich schnell gewesen.

## `_matplotlib()`

### *Zeile 226 · 13 Zeilen*

*kein Docstring*

## `_komma(FuncFormatter, stellen, vorzeichen)`

### *Zeile 241 · 13 Zeilen*

Deutsches Dezimalkomma auf den Achsen.

`stellen` ist nicht kosmetisch: Macro-F1 liegt zwischen 0,328 und 0,334 - mit zwei Nachkommastellen stunden an allen Achsenmarken dieselben "0,33", und die Abbildung waere sinnlos.

`vorzeichen` setzt auf Differenzachsen ein explizites Plus. Ohne das liest sich "2,5" wie ein Absolutwert statt wie ein Abstand.

### **`_prozent(FuncFormatter, stellen)`**

*Zeile 256 · 4 Zeilen*

Prozentachse mit deutschem Dezimalkomma.

### **`_text(pfad)`**

*Zeile 262 · 2 Zeilen*

*kein Docstring*

### **`_gepaarte_differenz()`**

*Zeile 267 · 32 Zeilen*

Je Verfahren die 10 Wiederholungsmittel der Differenz zur Baseline.

Gepaart wird auf (wiederholung, fold) - also auf identischen Testzeilen. Genau diese Paarung liegt auch dem Wilcoxon-Test in m02/m03 zugrunde; die Abbildung zeigt damit dieselbe Groesse, die getestet wird, und nicht eine andere, die zufaellig aehnlich aussieht.

### **`a1_gegen_baseline(plt, FuncFormatter)`**

*Zeile 301 · 54 Zeilen*

Die Primaeraussage: jedes Verfahren gegen seine Stufe-2-Baseline.

Dargestellt sind die 10 Wiederholungsmittel als Punkte und ihre Verteilung als Kasten. Ein Fehlerbalken waere hier die schlechtere Wahl: Bei zehn Werten zeigt der Punktschwarm die Verteilung selbst, statt sie durch eine Kennzahl zu ersetzen, die Symmetrie unterstellt.

Die Nulllinie ist die Baseline. Wo der Kasten sie schneidet, ist der Unterschied nicht gesichert - unabhaengig davon, was der Mittelwert sagt.

### **`a2_foldstruktur(plt, FuncFormatter)`**

*Zeile 358 · 43 Zeilen*

Warum in A1 gepaart wird: die Folds bewegen alle Verfahren gemeinsam.

Gezeigt wird eine einzelne Wiederholung, sonst waeren es 50 Linien. Die Aussage haengt nicht an der Auswahl - die uebrigen neun sehen genauso aus, was sich an der Streuungszerlegung in der Fusszeile ablesen laesst.

### **`_spezifikationszeilen()`**

*Zeile 404 · 36 Zeilen*

(Beschriftung, RMSE, Gruppe) fuer A3 - alles aus CSV, nichts von Hand.

### **a3\_spezifikation(plt, FuncFormatter)**

*Zeile 442 · 55 Zeilen*

Unterfrage 4: Was bewegt mehr - das Verfahren oder die Spezifikation?

Alle Werte sind RMSE auf `anzahl\_einsaetze`, gemittelt ueber dieselben 50 Laeufe. Sie sind damit unmittelbar vergleichbar; es ist kein Wechsel des Massstabs zwischen den Gruppen im Spiel.

Die Balken sind nach Gruppen sortiert, nicht global - sonst stuede die Referenz mitten zwischen den Verfahren und die Gruppierung waere nicht ablesbar.

### **a4\_laufzeit\_guete(plt, FuncFormatter)**

*Zeile 500 · 80 Zeilen*

Unterfrage 3: Aufwand gegen Guete.

Die Zeitachse ist logarithmisch, weil zwischen Ridge und den Ensembles Groessenordnungen liegen - linear waere Ridge ein Punkt auf der Null.

### **a5\_holdout(plt, FuncFormatter)**

*Zeile 583 · 61 Zeilen*

Die einmalige Auswertung auf den sechs zurueckgehaltenen Stadtteilen.

Anders als in A1 gibt es hier KEINE Streuung - das Hold-out wird genau einmal ausgewertet. Fehlerbalken waeren an dieser Stelle falsch; die Einmaligkeit ist der Zweck des Hold-outs.

Alle drei Stufen stehen nebeneinander, damit sichtbar bleibt, wovon der Abstand gemessen wird.

### **\_faktorgruppen\_balken()**

*Zeile 647 · 29 Zeilen*

(Strang, Beschriftung, Anteile je Gruppe) - alles aus CSV.

ZWEI QUELLEN, ZWEI GROESSEN - das ist der Grund fuer die Fusszeile der Abbildung. Der Mengenbalken zeigt standardisierte KOEFFIZIENTEN des Poisson-GLM, die Strukturbalken zeigen SHAP-BEITRAEGE. Beide sind auf Summe 1 normiert und damit nebeneinander lesbar, aber sie sind nicht dieselbe Groesse und duerfen nicht als eine gelesen werden.

Dass die Menge aus der Baseline kommt, ist kein Notbehelf: m04 ueberspringt dort jedes Vergleichsverfahren, weil keines seine Stufe-2-Baseline schlaegt - und Beitraege eines unterlegenen Modells waeren erklartes Rauschen. Das beste Modell des Mengenstrangs IST das Poisson-GLM.

### **a6\_faktorgruppen(plt, FuncFormatter)**

*Zeile 678 · 62 Zeilen*

UNTERFRAGE 1: Welche Faktorgruppe traegt wie viel?

Gestapelte Balken statt gruppierter: Die Anteile summieren sich je Modell auf 100 %, und genau diese Aufteilung ist die Aussage. Gruppierte Balken

wuerden zum Vergleich EINER Gruppe zwischen den Modellen einladen - das traegt hier nicht, weil die Werte aus verschiedenen Groessen stammen.

Die Segmente sind ueber die Schraffur unterschieden, nicht ueber den Grauwert. Fuenf Grautoene in einem Balken sind im Schwarzweissdruck nicht mehr sicher zu trennen.

#### **a7\_extrapolation(plt, FuncFormatter)**

##### *Zeile 743 · 65 Zeilen*

Warum manche Folds schwer sind - R-3 als Bild statt als Vorbehalt.

Ein Punkt je Lauf, 50 je Verfahren. Die drei Verfahren liegen bei gleichem  $x$  uebereinander, weil der Extrapolationsanteil eine Eigenschaft des FOLDS ist und nicht des Verfahrens - das ist kein Darstellungsfehler, sondern die halbe Aussage.

ABGRENZUNG ZU #34, dieselbe wie in `m04.extrapolation\_aufschluesseln`: Hier wird die Testmenge NICHT nach Extrapolationsgrad geschnitten und darin nach Verfahrensunterschieden gesucht. Die Einheit bleibt der Lauf, die Frage lautet, warum Laeufe unterschiedlich schwer sind. Die Primaeraussage bleibt unberuehrt.

#### **\_lage\_im\_suchraum(name, parameter, wert)**

##### *Zeile 811 · 30 Zeilen*

Relative Lage eines gefundenen Wertes in SEINEM Suchraum, 0 bis 1.

Ohne diese Normierung liessen sich die Parameter nicht in eine Abbildung bringen: `alpha` laeuft ueber sechs Zehnerpotenzen, `subsample` ueber 0,4 Einheiten. Die Frage lautet ohnehin nicht "welcher Wert", sondern "wie weit streuen die fuenf Folds in dem Raum, der zur Verfuegung stand".

Die Umrechnung spiegelt `m02.suchraum()`: loguniform wird logarithmisch normiert, `int` und `uniform` linear, `choice` ueber die Position in der Liste aus config\_modelle. Faellt ein Wert aus seinem Raum, gibt es None - dann hat sich der Suchraum seit dem Lauf geaendert, und die Zeile fehlt in der Abbildung, statt eine falsche Lage vorzutauschen.

#### **\_hyperparameter\_lagen()**

##### *Zeile 843 · 25 Zeilen*

Je (Strang, Verfahren, Parameter) die fuenf Fold-Werte, auf 0..1 normiert.

#### **a8\_hyperparameter(plt, FuncFormatter)**

##### *Zeile 870 · 83 Zeilen*

Wie stabil ist die Modellwahl bei 29 Entwicklungsstadtteilen?

Jede Zeile ist ein Hyperparameter, die fuenf Punkte sind die fuenf Folds. Die graue Strecke ist der volle Suchraum. Streuen die Punkte ueber die ganze Strecke, hat die Kreuzvalidierung diesen Parameter nicht bestimmt - das Tuning waehlt dann faktisch zufaellig.

Das ist eine Aussage fuer Kapitel 8 und keine Fehlermeldung: Bei 23 Trainingsstadtteilen je Fold ist genau dieses Verhalten zu erwarten, und es



ist der ehrlichere Umgang damit, es zu zeigen statt die fuenf Parameter-saetze nur zu mitteln.

Die Spannweite rechts ist die Kennzahl dazu: 1,00 heisst "von einem Rand des Suchraums zum anderen".

#### **a9\_parallelisierung(plt, FuncFormatter)**

##### *Zeile 956 · 73 Zeilen*

Die zweite Haelfte von Unterfrage 3: Was bringen zusaetzliche Kerne?

A4 traegt die EINKERNIGE Trainingszeit auf - das ist der Aufwand, der zwischen den Verfahren vergleichbar ist. Diese Abbildung zeigt die andere Groesse, die im selben Lauf miterhoben wurde: den Faktor, um den derselbe Fit ueber alle Kerne schneller wird.

Die Linie bei 1,0 ist nicht Dekoration. Ein Wert DARUNTER heisst, dass der parallele Fit LANGSAMER war - der Verwaltungsaufwand der Threads uebersteigt den Gewinn. Ohne die Linie liest man solche Balken als kleinen Gewinn.

Bei Ridge ist ein Wert um 1 zu erwarten: Eine geschlossene Loesung hat nichts zu verteilen. Auch das ist ein Ergebnis fuer Unterfrage 4 und kein Messfehler.

#### **a10\_qq\_residuen(plt, FuncFormatter)**

##### *Zeile 1032 · 57 Zeilen*

Halten die Residuen der linearen Spezifikation die Normalverteilung?

Schroeter hat den QQ-Plot am 10.08.2026 selbst genannt. Er beantwortet eine Frage, die ein Test allein nicht beantwortet: WO die Abweichung liegt. Jarque-Bera sagt nur, DASS die Verteilung nicht normal ist - bei  $n = 3.036$  sagt er das ohnehin fast immer.

Zu lesen ist die Abbildung an den ENDEN. Liegen die Punkte in der Mitte auf der Geraden und biegen nur aussen ab, ist die Verteilung im Kern normal und hat schwere Raender - genau das, was bei Einsatzzahlen zu erwarten ist. Eine Kruemmung ueber die ganze Laenge waere etwas anderes und schwerer wiegend.

WOZU DAS UEBERHAUPT GEZEIGT WIRD, obwohl Normalitaet fuer die Punktprognose nicht erforderlich ist (siehe die Tabelle in Abschnitt 6 der Eignungspruefung): Die Anforderung wird geprueft und die Antwort lautet "besteht hier nicht". Das ist eine Aussage. Sie ungeprueft zu lassen waere keine.

Gezeichnet, nicht gerechnet - die Quantile stehen fertig in  
`qq\_residuen.csv`, erzeugt von `v2\_eignung.annahmen()`.

#### **main()**

##### *Zeile 1092 · 34 Zeilen*

*kein Docstring*